

Q2.1**a) + b) + c):**

```
Pc = imread('mrt-train.jpg');
whos Pc
P = rgb2gray(Pc);

figure;
imshow(P)
min(P(:)), max(P(:))
```

Result:

```
ans =
uint8
13

ans =
uint8
204
```

Jerrell's comments: The low contrast image is displayed here as well as the min value being 13 and max value being 204.

d) and e):

```
P2 = imsubtract(P, double(min(P(:))));
P2 = immultiply(P2, 255 / double(max(P(:)) - min(P(:))));
min(P2(:)), max(P2(:))

figure;
imshow(P2)
```

Result:

```
ans =
uint8
0

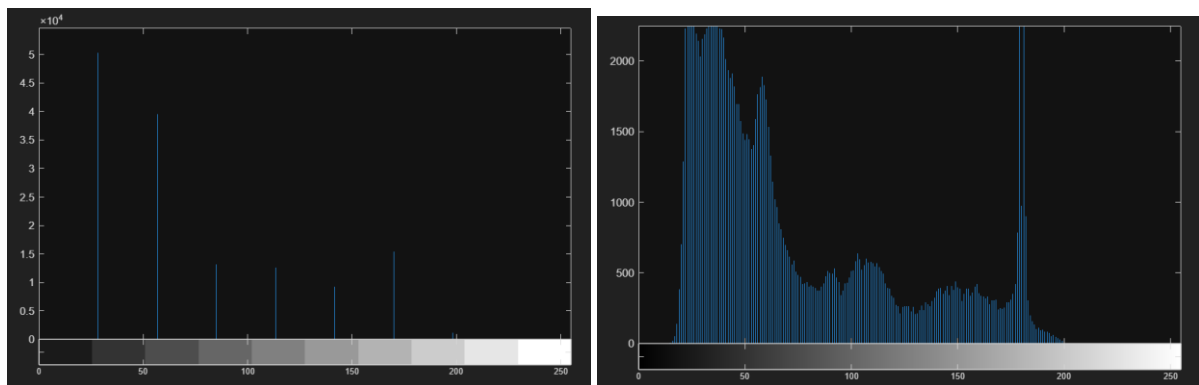
ans =
uint8
255
```

Q2.2

a)

```
figure;  
imhist(P,10);  
  
figure;  
imhist(P,256);
```

Result:

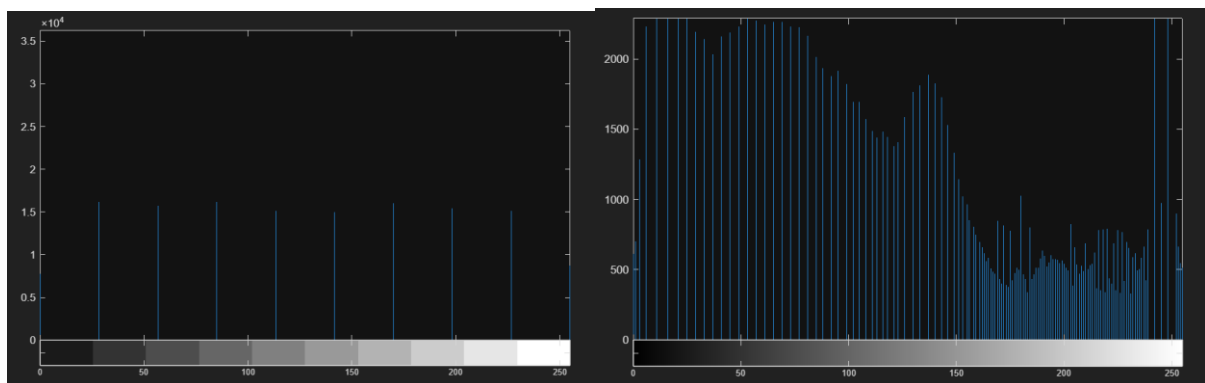


Jerrell's comments: The histogram with 256 bins has a more accurate representation of the distribution. Between 150 to 200 we see that the graph on the right (with 256 bins) is able to capture the large spike in frequency, while the graph on the left only shows a small rise in frequency.

b)

```
P3 = histeq(P,255);  
figure;  
imhist(P3,10);  
figure;  
imhist(P3,256);
```

Result:

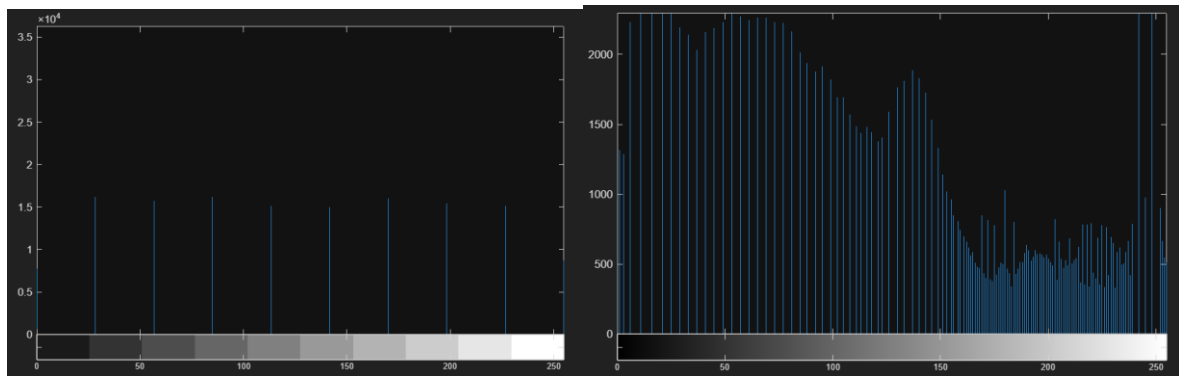


Jerrell's comments: The histogram with 10 bins on the left is a lot more equalized compared to the histogram on the right as the frequency levels are more uniform. Meanwhile the one on the right has a lot of high frequency bunched on the left and low frequencies bunched on the right.

c)

```
P3A = histeq(P3,255);  
figure;  
imhist(P3A,10);  
figure;  
imhist(P3A,256);
```

Result:



Q2.3

a)

```
sigma1 = 1.0;
sigma2 = 2.0;
n = 2;
[x, y] = meshgrid(-n:n, -n:n);

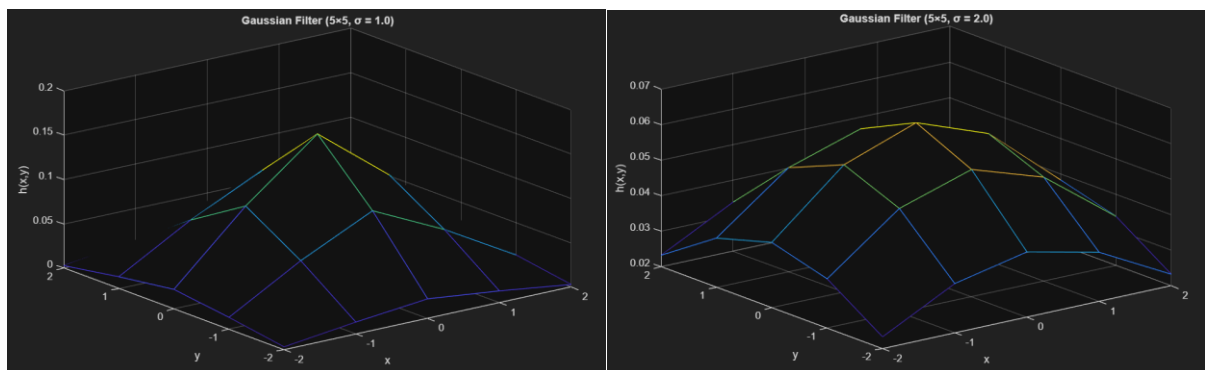
h1 = exp(-(x.^2 + y.^2) / (2 * sigma1^2)) / (2 * pi * sigma1^2);
h1 = h1 / sum(h1(:));

figure;
mesh(x, y, h1);
title('Gaussian Filter (5x5,  $\sigma = 1.0$ )');
xlabel('x'); ylabel('y'); zlabel('h(x,y)');

h2 = exp(-(x.^2 + y.^2) / (2 * sigma2^2)) / (2 * pi * sigma2^2);
h2 = h2 / sum(h2(:));

figure;
mesh(x, y, h2);
title('Gaussian Filter (5x5,  $\sigma = 2.0$ )');
xlabel('x'); ylabel('y'); zlabel('h(x,y)');
```

Result:



b)

```
lib_img = imread('lib-gn.jpg');
figure;
imshow(lib_img)
title('Original');
Igray = im2gray(lib_img);
Igray = im2double(Igray);

I_filt1 = conv2(Igray, h1, 'same'); %  $\sigma = 1.0$ 
I_filt2 = conv2(Igray, h2, 'same'); %  $\sigma = 2.0$ 

figure;
imshow(I_filt1);
title('Filtered ( $\sigma = 1.0$ )');

figure;
imshow(I_filt2);
title('Filtered ( $\sigma = 2.0$ )');
```

Result:



Jerrell's comments: The 2 filters are effective at reducing the noise, however the tradeoff is that the image quality becomes more blurred in the process.

c)

```
lib_img1 = imread('lib-sp.jpg');
figure;
imshow(lib_img1)
title('Original');
Igray1 = im2gray(lib_img1);
Igray1 = im2double(Igray1);

I_filt1 = conv2(Igray1, h1, 'same');    %  $\sigma = 1.0$ 
I_filt2 = conv2(Igray1, h2, 'same');    %  $\sigma = 2.0$ 

figure;
imshow(I_filt1);
title('Filtered ( $\sigma = 1.0$ )');

figure;
imshow(I_filt2);
title('Filtered ( $\sigma = 2.0$ )');
```

Result:



Jerrell's comments: The filters seem better at handling Gaussian noise compared to speckle noise, as we can still see the white speckles even after applying the filters.

Q2.4

```
I_med3 = medfilt2(Igray, [3 3]); % 3x3 neighbourhood
I_med5 = medfilt2(Igray, [5 5]); % 5x5 neighbourhood

figure;
imshow(Igray);
title('Original Image');

figure;
imshow(I_med3);
title('Median Filtered (3x3)');

figure;
imshow(I_med5);
title('Median Filtered (5x5)');

I_med3 = medfilt2(Igray1, [3 3]); % 3x3 neighbourhood
I_med5 = medfilt2(Igray1, [5 5]); % 5x5 neighbourhood
```

Result:

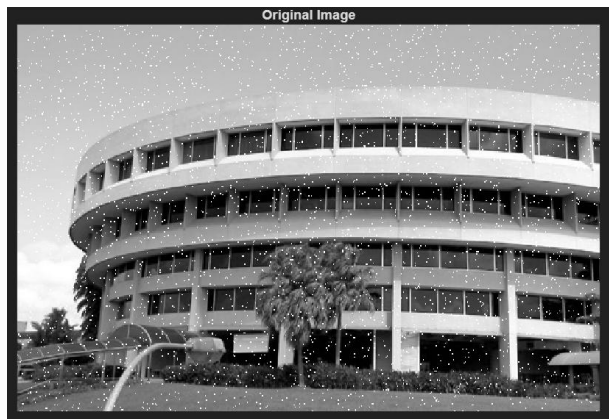


```
I_med3 = medfilt2(Igray1, [3 3]); % 3x3 neighbourhood
I_med5 = medfilt2(Igray1, [5 5]); % 5x5 neighbourhood

figure;
imshow(Igray1);
title('Original Image');

figure;
imshow(I_med3);
title('Median Filtered (3x3)');

figure;
imshow(I_med5);
title('Median Filtered (5x5)');
```



Jerrell's comments: Median filtering seems better suited for removing the white speckles, while Gaussian filtering is better suited for removing noise. The drawback of Gaussian filtering would be blurring the image, while drawback of median filtering is that the image appears to be smudged with a loss of key details.

Q2.5

a) + b) + c)

```
pck_img = imread('pck-int.jpg');
Igray2 = im2gray(pck_img);
Igray2 = im2double(Igray2);

figure;
imshow(Igray2);
title('Original image');

F = fft2(Igray2);
S = abs(F).^2;

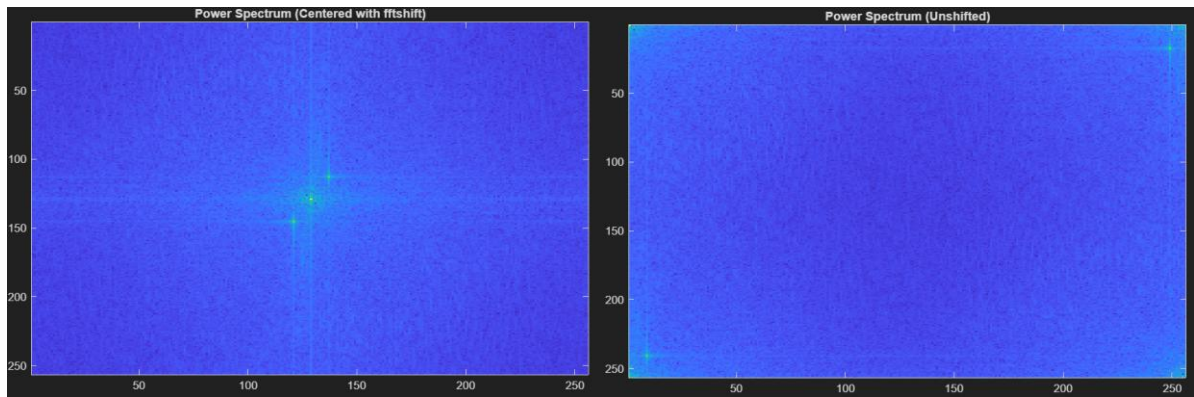
% Display Power Spectrum
figure;
imagesc(fftshift(S.^0.1));
colormap('default');
title('Power Spectrum (Centered with fftshift)');

figure;
imagesc(S.^0.1);
colormap('default');
title('Power Spectrum (Unshifted)');

disp('Click on the two bright interference peaks');
[cols, rows] = ginput(2);
peak_coords = round([rows cols]);
disp(peak_coords);
```

Result:





Click on the two bright interference peaks

241 9
18 249

d) + e)

```
F_filtered = F;

% Define size of neighborhood to zero out (5x5)
n = 2;

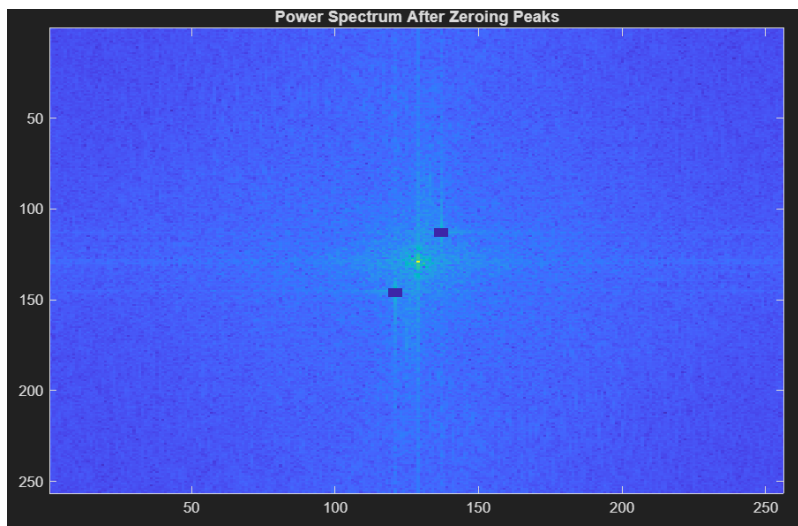
for k = 1:size(peak_coords,1)
    r = peak_coords(k,1);
    c = peak_coords(k,2);
    F_filtered(r-n:r+n, c-n:c+n) = 0;
end

S_filtered = abs(F_filtered).^2;

figure;
imagesc(fftshift(S_filtered.^0.1));
colormap('default');
title('Power Spectrum After Zeroing Peaks');

I_filtered = real(ifft2(F_filtered));
figure;
imshow(I_filtered, []);
title('Image After Bandpass Filtering (Interference Suppressed)');
```

Result:





Jerrell's comments: After we remove the high frequency regions on the graph, the image looks a lot more visible and clear without the lines. However, it still doesn't seem to be perfect image yet as we can still see some of the black lines especially near the edges of the image. Removing a larger rectangle from the graph may result in a better result.

f)

```
pck_img = imread('primate-caged.jpg');
Igray3 = im2gray(pck_img);
Igray3 = im2double(Igray3);

figure;
imshow(Igray3);
title('Original image');

F = fft2(Igray3);
S = abs(F).^2;

figure;
imagesc(S.^0.1);
colormap('default');
title('Power Spectrum (Unshifted)');

disp('Click on the two bright interference peaks');
[cols, rows] = ginput(2);
peak_coords = round([rows cols]);
disp(peak_coords);

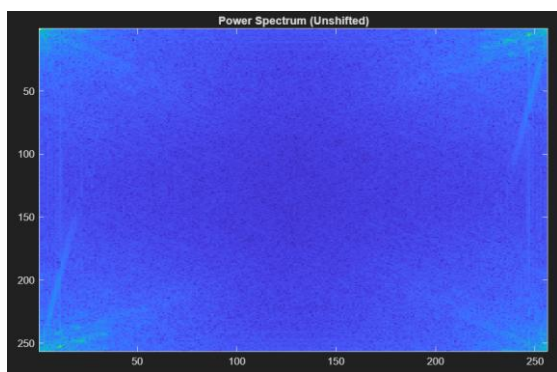
F_filtered = F;
|
% Define size of neighborhood to zero out (5x5)
n = 2;

for k = 1:size(peak_coords,1)
    r = peak_coords(k,1);
    c = peak_coords(k,2);
    F_filtered(r-n:r+n, c-n:c+n) = 0;
end

S_filtered = abs(F_filtered).^2;

I_filtered = real(ifft2(F_filtered));
figure;
imshow(I_filtered, []);
title('Image After Bandpass Filtering (Interference Suppressed)');
```

Result:



Q2.6

```
lib_img2 = imread('book.jpg');
Igray2 = im2gray(lib_img2);
Igray2 = im2double(Igray2);

figure;
imshow(Igray2)

disp('Click on the 4 corners');
[X Y] = ginput(4);
disp([X, Y])
```

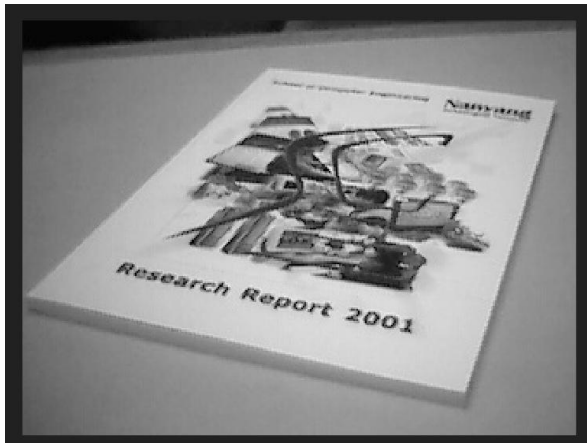
```
x1=0; y1=0;
x2=210; y2=0;
x3=210; y3=297;
x4=0; y4=297;

v = [x1; y1; x2; y2; x3; y3; x4; y4];

A = [X(1) Y(1) 1 0 0 0 -x1*X(1) -x1*Y(1);
     0 0 0 X(1) Y(1) 1 -y1*X(1) -y1*Y(1);
     X(2) Y(2) 1 0 0 0 -x2*X(2) -x2*Y(2);
     0 0 0 X(2) Y(2) 1 -y2*X(2) -y2*Y(2);
     X(3) Y(3) 1 0 0 0 -x3*X(3) -x3*Y(3);
     0 0 0 X(3) Y(3) 1 -y3*X(3) -y3*Y(3);
     X(4) Y(4) 1 0 0 0 -x4*X(4) -x4*Y(4);
     0 0 0 X(4) Y(4) 1 -y4*X(4) -y4*Y(4)
     ];

u = A \ v;

U = reshape([u;1], 3, 3)';
disp('Matrix U:');
disp(U);
w = U*[X'; Y'; ones(1,4)];
w = w ./ (ones(3,1) * w(3,:));
disp('Transformed points w:');
disp(w);
```



```
Click on the 4 corners
144.1053  29.8738
307.1414  48.5455
254.3140  215.6803
  3.8397  160.1205

Matrix U:
  1.5093    1.6254 -266.0616
 -0.4380    3.8248  -51.1378
  0.0002    0.0055   1.0000

Transformed points w:
   0  210.0000  210.0000  -0.0000
   0  -0.0000  297.0000  297.0000
  1.0000  1.0000  1.0000  1.0000
```

Jerrell's comments: As we see from transformed points w, I was able to get back the 4 corners of the desired image (which is given by my x1 y1, x2 y2, x3 y3 and x4 y4 values)

```
T = maketform('projective', U);
P2 = imtransform(Igray2, T, 'XData', [0 210], 'YData', [0 297]);
figure;
imshow(P2);
```



Jerrell's comments: This is what I expected. The quality will definitely not be so good because in the original image, the words at the top don't have good quality anyway. The image is also stretched out so there will be some loss in the key details.

```
figure;  
imshow(P2);  
hold on;  
  
x = 135;  
y = 145;  
w = 50;  
h = 50;  
  
rectangle('Position', [x, y, w, h], ...  
          'EdgeColor', 'magenta', ...  
          'LineWidth', 2);  
  
hold off;
```



Q2.7

Algorithm 1:

```
% Each row of X is a feature vector (including bias term as first element)
X = [3 3 1; % x1
     1 1 1]; % x2

% Class labels: +1 for C1, -1 for C2
y = [ 1; -1];

alpha = 1;
[num_samples, num_features] = size(X);
w = zeros(num_features, 1);

max_epochs = 100;
for epoch = 1:max_epochs
    num_errors = 0;
    for k = 1:num_samples
        xk = X(k, :);
        yk = y(k);

        decision = w' * xk;

        % Misclassified C1
        if yk == 1 && decision <= 0
            w = w + alpha * xk;
            num_errors = num_errors + 1;

        % Misclassified C2
        elseif yk == -1 && decision >= 0
            w = w - alpha * xk;
            num_errors = num_errors + 1;
        end
    end

    fprintf('Epoch %d: Number of errors = %d\n', epoch, num_errors);

    if num_errors == 0
        break;
    end
end

disp('Final weight vector:');
disp(w);
disp('Final equation: y = x1 + x2 -3');
```

Result:

```
Epoch 1: Number of errors = 2
Epoch 2: Number of errors = 1
Epoch 3: Number of errors = 1
Epoch 4: Number of errors = 2
Epoch 5: Number of errors = 1
Epoch 6: Number of errors = 0
Final weight vector:
     1
     1
    -3
Final equation: y = x1 + x2 -3
```

Jerrell's comments: It only took 6 episodes to obtain the final weight vectors so it is pretty efficient. However, this may easily result in the overfitting problem since it is a more naïve approach.

Algorithm 2:

```
% Each row of X is a feature vector (bias + features)
X = [3 3 1; % x1
     1 1 1]; % x2

% Target outputs: +1 for class 1, -1 for class 2
r = [ 1; -1];

alpha = 0.1;
[num_samples, num_features] = size(X);
w = zeros(num_features, 1);

max_epochs = 150;
for epoch = 1:max_epochs
    total_error = 0;
    for k = 1:num_samples
        xk = X(k, :);
        rk = r(k);
        yk = w' * xk;
        err = rk - yk;
        w = w + alpha * err * xk;
        total_error = total_error + err^2;
    end

    fprintf('Epoch %d: Total squared error = %.4f\n', epoch, total_error);

    % Stopping criterion
    if total_error < 1e-4
        break;
    end
end

disp('Final weight vector:');
disp(w);
disp('Final equation: r = 0.5x1 + 0.5x2 - 2');
```

Result:

```
Epoch 95: Total squared error = 0.0003
Epoch 96: Total squared error = 0.0002
Epoch 97: Total squared error = 0.0002
Epoch 98: Total squared error = 0.0002
Epoch 99: Total squared error = 0.0002
Epoch 100: Total squared error = 0.0002
Epoch 101: Total squared error = 0.0001
Epoch 102: Total squared error = 0.0001
Epoch 103: Total squared error = 0.0001
Epoch 104: Total squared error = 0.0001
Epoch 105: Total squared error = 0.0001
Final weight vector:
    0.4976
    0.4976
   -1.9887
Final equation: r = 0.5x1 + 0.5x2 - 2
```

Jerrell's comments: As we can see, algorithm 2 took a lot more episodes (105) to obtain the final weight vector, however it will be more accurate in distinguishing the right and wrong class because there is less overfitting to the data.